

Name: Ethan Jia
Student ID: ethanjia (2235723)

Part 1

Public Shareable Link: <https://studio.edgeimpulse.com/public/1006629/live>

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 38 ms, inference 573 us, anomaly 0 ms, postprocessing 48 us
#Classification predictions:
  idle: 0.914062
  lift: 0.085937
```

Fig. 1 Screenshot of the model correctly identifies idle state

```
Starting inferencing in 2 seconds...
Sampling...
Timing: DSP 39 ms, inference 574 us, anomaly 0 ms, postprocessing 49 us
#Classification predictions:
  idle: 0.000000
  lift: 0.996094
```

Fig. 2 Screenshot of the model correctly identifies lift state

Task 2

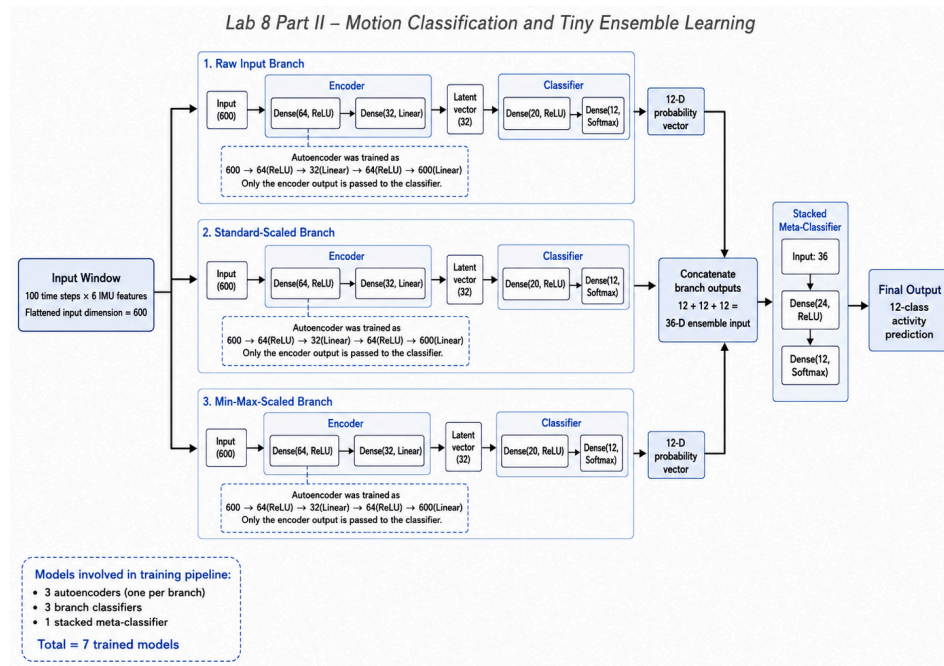


Fig. 3 Ensemble Pipeline Diagram

Question 2

This notebook compresses the models for TinyML deployment by applying pruning, quantization-aware training, and full integer TFLite conversion. The goal is to reduce the model size and make the final models more suitable for deployment on a resource-constrained device such as the Arduino Nano 33 BLE Sense.

First, pruning is applied using TensorFlow Model Optimization's `prune_low_magnitude()` method. The notebook uses a polynomial decay sparsity schedule with an initial sparsity of 10% and a final sparsity of 80%. The pruning schedule begins at step 0 and ends at step 500. During pruning, the model is trained for 5 epochs while the pruning callback updates the pruning step. This gradually forces many low-magnitude weights toward zero, creating a sparse model.

After pruning, the notebook strips the pruning wrappers before applying QAT. This is done using `strip_pruning()`. The pruning wrappers are useful during training because they apply and update pruning masks, but they are not part of the final model architecture. Stripping them produces a cleaner Keras model that keeps the pruned weights while removing the extra pruning logic. This step is necessary before QAT because the quantization tool expects a normal Keras model structure.

QAT is then applied to the stripped model using `quantize_model()`. Quantization-aware training simulates the effect of low-precision integer arithmetic during training, so the model can adapt to quantization noise before being converted to an int8 TFLite model. In this notebook, QAT is performed for 5 epochs after pruning.

The notebook also uses an additional mask-enforcement step during and after QAT. This is important because a simple pruning-then-QAT pipeline may allow some previously pruned weights to become nonzero again during QAT fine-tuning. The notebook extracts masks from the stripped pruned model, where zero weights indicate pruned connections. Then, during QAT, a custom callback repeatedly reapplies these masks so that pruned weights stay zero. This helps preserve the sparsity created during pruning while still allowing the remaining nonzero weights to adapt to quantization.

Finally, the QAT model is converted into a full integer int8 TFLite model. The converter uses `tf.lite.Optimize.DEFAULT`, sets the supported operations to `TFLITE_BUILTINS_INT8`, and sets both the inference input type and output type to int8. A representative dataset is provided during conversion. This representative data is needed so the TFLite converter can observe typical input ranges and choose appropriate scale and zero-point values for quantizing activations and tensors. Without representative data, the int8 model may have poor calibration and lower accuracy.

Overall, pruning reduces the number of effective nonzero weights, which can reduce storage requirements and potentially improve efficiency. Int8 quantization stores weights and activations using 8-bit integers instead of 32-bit floating-point values, which greatly reduces model size and

memory use. These compression methods are especially important for the Arduino Nano 33 BLE Sense because it has limited flash memory, RAM, and compute resources compared with a laptop or cloud environment.

Question 3

```
-----
Samples collected: 25
Samples collected: 50
Samples collected: 75
Samples collected: 100
Window collected. Running ensemble inference...
Raw model scores: 0.0000, 0.9102, 0.0000, 0.0000, 0.0000, 0.0820, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039, 0.0039
Standard-scaled model scores: 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.9961, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000
Min-max-scaled model scores: 0.0000, 0.1133, 0.1445, 0.0000, 0.7422, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0039
Meta-classifier scores: 0.0039, 0.1055, 0.1055, 0.0039, 0.0039, 0.5742, 0.0000, 0.0000, 0.1953, 0.0039, 0.0039, 0.0000
-----
Final prediction: Waist bends forward
Class index: 5
Confidence score: 0.5742
-----
```

Fig. 4 Screenshot of the Serial Monitor Output

For the Arduino deployment test, I used the provided Arduino sketch with the quantized ensemble models generated from the notebook. I placed the Arduino Nano 33 BLE Sense flat on a stable surface and opened the serial monitor at 115200 baud. The sketch successfully collected a 100-sample IMU window and then ran the three branch models followed by the stacked meta-classifier. The screenshot shows the raw model scores, standard-scaled model scores, min-max-scaled model scores, and the final meta-classifier output.

In this test, the final prediction was “Waist bends forward”, with class index 5 and a confidence score of 0.5742. Since the board was resting flat on the table, this label does not physically match the real test condition. However, the output is still acceptable as a deployment functionality test because the Arduino was able to collect live IMU data, run the three quantized branch models, combine their outputs, and produce a final prediction using the stacked meta-classifier.

The unexpected label is likely caused by a mismatch between the training data and the real deployment setup. The model was trained using the mHealth human activity dataset, while the deployed test used a stationary Arduino board on a table. The training data likely came from sensors worn on the body, so the sensor orientation, placement, motion patterns, and gravity direction are different from the Arduino test setup. Even when the board is not moving, the accelerometer still measures gravity, and small sensor noise can affect the 100-sample input window. There could also be some mismatch between the scaling parameters used in training and the live IMU values measured by the board.

To improve deployment reliability, one practical change would be to collect calibration data directly from the Arduino Nano 33 BLE Sense in the same orientation used during deployment and include a true idle/resting-board class during training. Another improvement would be to apply smoothing or majority voting across several consecutive prediction windows, so that one uncertain or noisy window does not immediately determine the final predicted activity.

Question 4

During testing, the Arduino model responded to changes in board motion. When the board was placed flat and still, the model predicted “Waist bends forward” with a confidence score of 0.5742. When I moved the board, the prediction changed to other activity labels such as “Jump front and back” with confidence 0.4531 and “Frontal elevation of arms” with confidence 0.5391. This shows that the deployed model is reacting to live IMU changes, but the predicted labels do not always match the actual motion performed with the board.

The main issue is likely a mismatch between the training data and the real deployment setup. The model was trained on the mHealth human activity dataset, which likely used body-worn IMU sensors during human activities. In contrast, the Arduino Nano 33 BLE Sense was handheld or placed on a table, so the sensor orientation, placement, gravity direction, and motion patterns were different. As a result, the model may classify the live Arduino movement as the closest activity pattern it learned during training, even if the label is not physically accurate.

Other possible causes include IMU noise, small differences in sensor units or ranges, and scaling mismatch between the notebook preprocessing and the Arduino sketch. To improve reliability, I would collect training data directly from the Arduino in the same orientation and motion setup used during deployment. I would also add smoothing or majority voting across several prediction windows to reduce unstable label changes.